COMP3009 - Data Mining
Final Assignment

Henry (23028920)

Final average classification accuracy:

83.57%

Summary

For this COMP3009 assignment, I am tasked with investigating and implementing a supervised machine learning classification model to predict whether network activity is normal or malicious. The training dataset contained 145,853 observations, with approximately 73.41% labelled as malicious.

Data preparation was performed on the data, including applying preprocessing steps to fix data problems such as missing values, duplicate observations and attributes, feature redundancy, numerical scaling and categorical encoding. Three classifiers were implemented and compared: k-Nearest Neighbours (k-NN), Decision Tree, and Naïve Bayes. Using stratified 10-fold cross-validation, their hyperparameters were evaluated and tuned.

The Decision Tree and k-NN classifiers achieved high cross-validation accuracy, with results close to 99.4%. A seven-neighbour k-NN was selected as the final model to be used on the test data. However, the final test performance was considerably lower, with an average accuracy of approximately 83.55% across the two test datasets.

Key findings from this investigation were that the training dataset contained highly predictive feature relationships, enabling classifiers to achieve strong validation performance. However, these models could not generalise to the test data. This demonstrates the importance of distinguishing cross-validation performance from generalisation to unseen observations.

The main lesson I learnt from this assignment was that preprocessing, model complexity, and high validation performance alone do not guarantee reliable performance across different datasets.

Methodology – Data Preparation

Dataset description

Provided along with the assignment were four Excel files: Feature-description.xlsx, Training-data.xlsx, Test-data-1.xlsx, and Test-data-2.xlsx.

Training-data.xlsx contains 145,854 rows and 43 columns, including the header row. This is the dataset that will be used to evaluate different classifiers, tune hyperparameters, and for the final training of the model. Feature-description.xlsx contains a brief description for each attribute. It will be used during preprocessing to assess the relevance of attributes. Test-data-1.xlsx and Test-data-2.xlsx are datasets that will be used only to evaluate the final model. Test-data-1.xlsx contains 59,807 rows, and Test-data-2.xlsx contains 58,021 rows, both including their header row. Both datasets have 43 columns, identical to those in Training-data.xlsx.

The training and test datasets both contain the boolean target variable, label. This variable indicates whether the internet activity entry is malicious, with 1 being malicious and 0 being normal.

In Training-data.xlsx, ~73% of entries are labelled as malicious and ~27% are labelled as normal internet activities.

Irrelevant attributes

Upon investigation, I have determined that the 'tcprtt' attribute is irrelevant. This is because it is simply the sum of two other attributes, 'synack' and 'ackdat'. This is confirmed through the similarity in their statistical properties, which can be seen in Table 1 and Figure 1, where their shapes are not too different, with the shapes in Figure 1 even sharing the same key features. Therefore, tcprtt will be dropped as it will provide no meaningful contribution.

Table 1. Comparing the mean, max, and min of 'tcprtt' and the sum of 'synack' and 'ackdat'.

	tcprtt	**synack + ackdat**
Mean	0.0365470	0.0365498
Max	1.482307	1.512073
Min	0.0	0.0

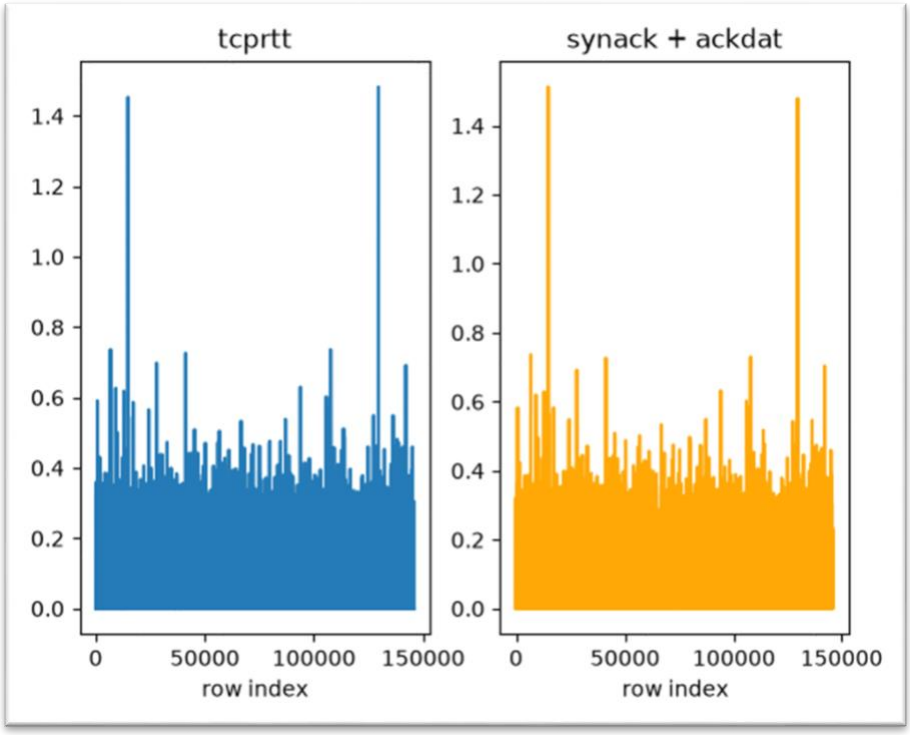


Figure 1. Comparison of values between 'tcprtt' and the sum of 'synack' and 'ackdat'.

Missing values

While there are no obvious missing values in the dataset, a closer investigation was conducted for 'service'. This is mainly due to the categorical attribute being able to take '-' values, and the investigation aimed at determining whether these can be considered missing values, as '-' accounts for ~47% of values in 'service'. Table 2 aims to establish whether there is a connection between 'service' being '-' and the target variable, and it is evident that there is no strong correlation between them; the proportions of 'label' are quite similar in both, and they are also similar to the overall class balance. Therefore, keeping or removing the attribute will not heavily affect the model. I have removed the attribute.

Table 2. Investigation of the correlation between '-' in 'service' and the value of the target variable.

'service' is '-'	Value of 'label'	Proportion
False	1	0.787250
False	0	0.212750
True	1	0.676226
True	0	0.323774

Duplicate instances

Using pandas' .duplicated function, it was determined that there are 355 duplicate rows and 2 duplicate columns in the dataset. The 355 duplicated rows make up a minute proportion of the entire dataset, and it was hard to determine whether they are erroneously duplicated or were repeated. Still, valid entries, removing them would not yield significant benefits, which is why I have left them in. However, having duplicate columns, 'is_ftp_login' and 'ct_ftp_cmd', means that all 145,853 values in both columns are identical, and that one of them contributes nothing to the classification model. Therefore, I have removed 'is_ftp_login'.

Data types

The datasets all contain three categorical attributes: proto, service, and state. 'proto' takes on 133 different values, 'service' takes on 13, and 'state' takes on 9. As for all of the classifiers in consideration- kNN, Decision Tree, and Naïve Bayes- they only take numerical inputs; I must convert these values into the eligible data types. Using one-hot encoding, I have converted 'proto' into a vector of length 133, 'service' into a vector of length 13, and 'state' into a vector of length 9.

Scaling and standardisation

Through a simple statistical analysis of the training dataset, it is clear that the features can take a wide range of values, with the maximum ~6,033,838,853 and the minimum 0. As evident in Table 3, the means of the attributes vary widely from one another, and this would ultimately affect the final predictive model, as the statistically larger features would dominate the smaller features. As we do not know how much each feature should contribute to the final property, we will normalise all values using Z-score normalisation, yielding relative values, as shown in Table 4, with respective max and min values of 209 and -1.7.

Table 3. Statistical properties of a sample of features from the training dataset.

Feature	Mean	Min	Max
sttl	176.80	0.00	255.00
dttl	75.21	0.00	254.00
sload	70518462.40	0.00	6033838853.94
dload	741966.43	0.00	22296225.18
sloss	4.94	0.00	4803.00
dloss	7.10	0.00	3246.00
sinpkt	628.82	0.00	84809.90
dinpkt	78.71	0.00	52517.03
sjit	4408.87	0.00	1206483.02

djit	641.84	0.00	291417.72
swin	114.24	0.00	255.00
stcpb	961895839.26	0.00	4294958913.00

Table 4. Statistical properties of a sample of features from the training dataset, but with Z-score normalisation.

Feature	Mean	Min	Max
sttl	-0.00	-1.70	0.75
dttl	0.00	-0.70	1.66
sload	0.00	-0.39	33.10
dload	0.00	-0.29	8.48
sloss	0.00	-0.08	74.47
dloss	-0.00	-0.17	79.54
sinpkt	0.00	-0.11	14.81
dinpkt	-0.00	-0.09	58.52
sjit	0.00	-0.11	30.90
djit	0.00	-0.15	68.27
swin	-0.00	-0.90	1.11
stcpb	-0.00	-0.71	2.47

Feature selection and engineering

As shown in Table 5, by computing correlation coefficients between features, I identified pairs to investigate further and decided whether to keep or remove them. Through the careful analysis of the statistical properties between each pair, I decided to remove 'dwin', which is essentially a duplicate of 'swin', sharing almost identical mean and standard deviation. While the other attributes also showed high correlation, their statistical differences were large enough to prevent their removal.

Table 5. Feature pairs with a correlation coefficient higher than 0.95.

Feature 1	Feature 2	Correlation Coefficient
ct_ftp_cmd	is_ftp_login	1.000000
dwin	swin	0.999965
sloss	sbytes	0.997310
dloss	dbytes	0.993889
ct_srv_dst	ct_srv_src	0.981141
ct_srv_dst	ct_dst_src_ltm	0.974134
sloss	spkts	0.971314
ct_dst_src_ltm	ct_srv_src	0.969176
dloss	dpkts	0.968572
ct_src_dport_ltm	ct_dst_ltm	0.967416
synack	tcprtt	0.964176
sbytes	spkts	0.963762

ackdat	tcprtt	0.960973
dbytes	dpkts	0.955898

Further feature selection was done on ‘smean’ and ‘dmean’. This is primarily due to ‘smean’ being the product of ‘sbytes’ and ‘spkts’, and ‘dmean’ being the product of ‘dbytes’ and ‘dpkts’. As they are both proportional to their two factors, we can remove ‘smean’ and ‘dmean’.

No feature engineering was required, as my knowledge of internet activity isn't sufficient.

Class balance and instance handling

While no class balancing was done for the target variable, I have used pandas’ .skew() function to check the skewness of different features and found that the features of ‘trans_depth’, ‘response_body_len’, ‘sbytes’, ‘sloss’, ‘spkts’, and ‘dinpkt’ all have high skewness. Therefore, I applied a log transformation to these values specifically. No instance handling was done, as no data imputation, weighting, or selection was necessary.

This is because it could alter the training distribution and discard potentially informative records. Stratified cross-validation was used instead to preserve the class proportions.

Methodology – Data Classification

Classifier candidates

The three candidate supervised learning algorithms that were selected for this assignment are k-Nearest Neighbours (k-NN), Decision Tree, and Naïve Bayes. Each classifier was trained to predict whether a given network activity was malicious (label 1) or normal (label 0).

k-Nearest Neighbours (k-NN). This classifier predicts the label of an observation based on the majority class among its k nearest training observations. k-NN was selected because it is suitable for classifying numerical features, and the network already consists mostly of numerical vectors, with only a few requiring conversion. The number of selected neighbours and the feature scales must be considered when using k-NN.

Decision Tree (DT). This classifier recursively partitions the observations through feature-based decision rules. Decision trees are often used for their interpretability and ability to model non-linear patterns and relationships, which is very beneficial for network activities, where features often lack linear relationships. However, the model's complexity must be accounted for to prevent overfitting.

Naïve Bayes (NB). This classifier estimates class probabilities using Bayes' theorem, where conditional independence is assumed between features of the given class. This classifier provides a more computationally efficient baseline, although its assumptions may be violated when attributes are correlated.

All classifiers were evaluated using the same preprocessed training dataset and cross-validation strategy to enable comparison.

Cross-validation strategy

A 10-fold Stratified K-Fold cross-validation strategy was used, with shuffling enabled and a random seed of 67. Stratification was primarily selected due to the class imbalance in the dataset, with ~73% malicious and ~27% normal activities. Through preserving these proportions across folds, we reduce the risk of obtaining unrepresentative partitions. Each iteration involved nine folds being used for training and the remaining fold for validation. Each observation, therefore, only contributed to validation once.

Learned preprocessing steps such as scaling, standardisation, and one-hot encoding were all implemented into scikit-learn pipelines. This ensured that no validation information leaked into the training data for each fold.

The performance of the classifiers was assessed using metrics such as accuracy, precision, recall, F1 Score, and confusion matrices. By considering all these metrics, I evaluated the advantages and disadvantages of each classifier to select the final model.

Hyperparameter tuning

Hyperparameters for each classifier were evaluated using cross-validation on the training dataset.

K-Nearest Neighbours. The k parameter controls the number of nearby observations considered during prediction. The values of k = 1, 3, 7, 11, 17, and 21 were considered. The best mean cross-validation accuracy was achieved with k = 7, at ~99.3%.

Decision Tree. The two parameters under consideration were the splitting criterion and the minimum number of samples required to split. Gini impurity and entropy were the splitting criteria investigated. The best configuration found was entropy with min_samples_split = 2, achieving ~ 99.4% cross-validation accuracy.

Naïve Bayes. The var_smoothing parameter was under-investigated for this classifier. Values ranging from 10^{-6} to 1 were tested, and the highest cross-validation accuracy of ~98.2% was obtained using var_smoothing = 0.1.

Classifier evaluation

Table 6—the final hyperparameters selected for each classifier and their mean cross-validation accuracy.

Classifier	Hyperparameters	Mean CV accuracy
k-NN	k = 7	99.3055%
Decision Tree	Gini, min_samples_split = 2	99.4302%
Naïve Bayes	var_smoothing = 0.1	98.2105%

All of the above models achieved nearly identical accuracy, with the k-NN and Decision Tree outperforming k-NN by approximately 1.2%.

Table 7. Comparison of the three classification models using various metrics

Model	Classification	Precision	Recall	F1-Score	Support
k-NN	Normal	1.00	0.98	0.99	38,777
	Malicious	0.99	1.00	1.00	107,077
	Accuracy	—	—	0.99	145,854

	Macro Avg	0.99	0.99	0.99	145,854
	Weighted Avg	0.99	0.99	0.99	145,854
DT	Normal	0.99	0.99	0.99	38,777
	Malicious	1.00	1.00	1.00	107,077
	Accuracy	—	—	0.99	145,854
	Macro Avg	0.99	0.99	0.99	145,854
	Weighted Avg	0.99	0.99	0.99	145,854
NB	Normal	0.97	0.96	0.97	38,777
	Malicious	0.98	0.99	0.99	107,077
	Accuracy	—	—	0.98	145,854

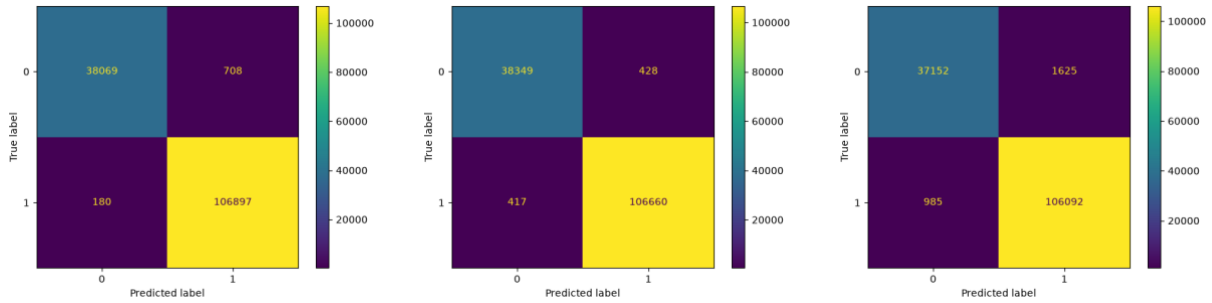


Figure 2. The three confusion matrices. From left to right: k-NN, Decision Tree, Naive Bayes.

In terms of network activity, a false negative is when the system fails to detect a legitimate threat/malicious activity, and a false positive is when the system mistakes normal activity for malicious activity. Minimising false negatives is especially important and arguably more so than minimising false positives.

As shown in Figure 2, k-NN has the fewest false negatives, whereas Decision Tree has the fewest false positives. Naïve Bayes underperforms in both categories compared to the rest.

Prediction Results

Comparison of the two best classifiers

Table 8. Comparison of k-NN and Decision Tree using various metrics.

Metric	k-NN	Decision Tree
Mean CV Accuracy	0.9930	0.9943
Malicious-class Precision	0.9934	0.9960
Malicious-class Recall	0.9983	0.9961
Malicious-class F1-score	0.9959	0.9961
Test Accuracy	0.9939	0.9942

Model selection

The two highest-performing classifiers were k-NN and Decision Tree. Their cross-validation accuracies differed by around 0.1%, suggesting similar performance under the current validation procedure.

Although the Decision Tree is marginally higher, I have selected k-NN with k = 7 as my final classifier. This is because, while its performance was effectively the same, k-NN has a higher recall for malicious activity (0.9983) than the Decision Tree (0.9961). This corresponds to a lower false-negative rate, reducing the likelihood of malicious activity being classified by the system as normal. This trade-off would be the most beneficial for systems where malicious traffic can have more severe consequences than normal traffic being flagged as malicious.

Final test performance

The final preprocessing pipeline consisted of all the datasets being preprocessed with the steps mentioned above. This includes removing certain columns, standardising numerical attributes, and encoding categorical attributes. With the pipeline and hyperparameters selected, the classifier will be fitted using the entire Training-data.xlsx. The two independent datasets were then used to generate the final predictions, which are used to calculate the final accuracy.

Table 9—final accuracies for the test datasets.

Evaluation dataset	Accuracy
Test-data-1.xlsx	0.7804604812145736
Test-data-2.xlsx	0.8909532755381672
Average	0.83570688

As shown in Table 9, the final average accuracy was approximately 83.5%, considerably lower than the 99.4% accuracy estimated during cross-validation on the training data. These findings illustrate how the accuracies calculated from the training-data validation were not fully representative of the model's ability to generalise to unseen data.

References

Boriharn K. (10 July 2023). Visualising 3 Sklearn cross-validation: K-fold, shuffle and split, and time series split. Medium. https://medium.com/data-science/visualizing-sklearn-cross-validation-k-fold-shuffle-split-and-time-series-split-a13221eb5a56

Brownlee, J. (4 October 2023). A gentle introduction to k-fold cross-validation. Machine Learning Mastery. https://machinelearningmastery.com/k-fold-cross-validation/

Curtin University. (2026). COMP3009 Data Mining: Lecture and laboratory materials.

OpenAI. (2026). ChatGPT [Large language model]. https://chatgpt.com/

pandas development team. (n.d.). pandas documentation. https://pandas.pydata.org/docs/

scikit-learn developers. (n.d.-a). Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html

scikit-learn developers. (n.d.-b). Cross-validation: Evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html

scikit-learn developers. (n.d.-c). Pipelines and composite estimators. https://scikit-learn.org/stable/modules/compose.html